Perceptron Learning Algorithm

Recap

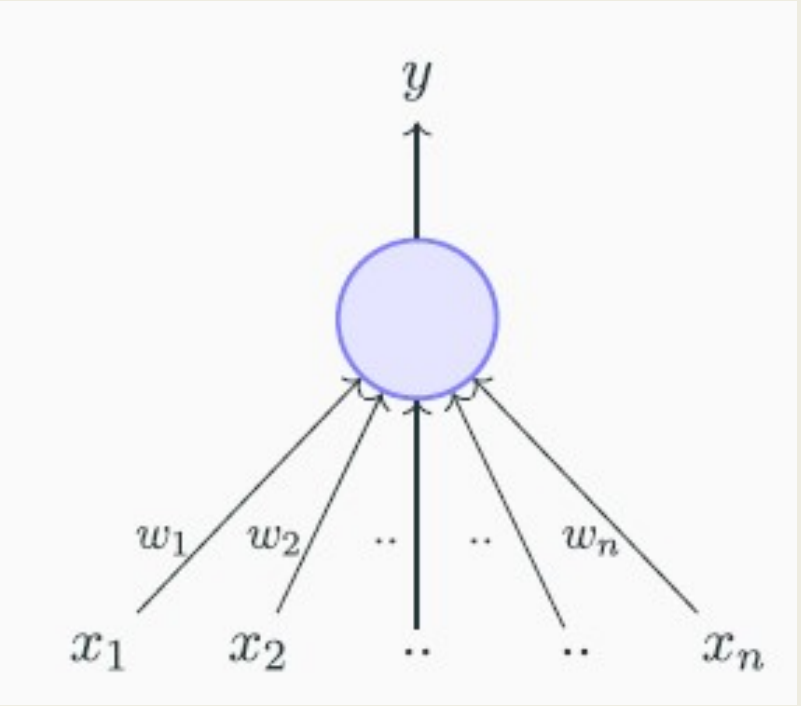
$$y = 1 \quad if \sum_{i=1}^{n} w_i * x_i \geq \theta$$

$$y = 0 \quad if \sum_{i=1}^{n} w_i * x_i < \theta$$

Rewriting the above

$$y = 1 \quad if \sum_{i=1}^{n} w_i * x_i - \theta \geq 0$$

$$y = 0 \quad if \sum_{i=1}^{n} w_i * x_i - \theta < 0$$

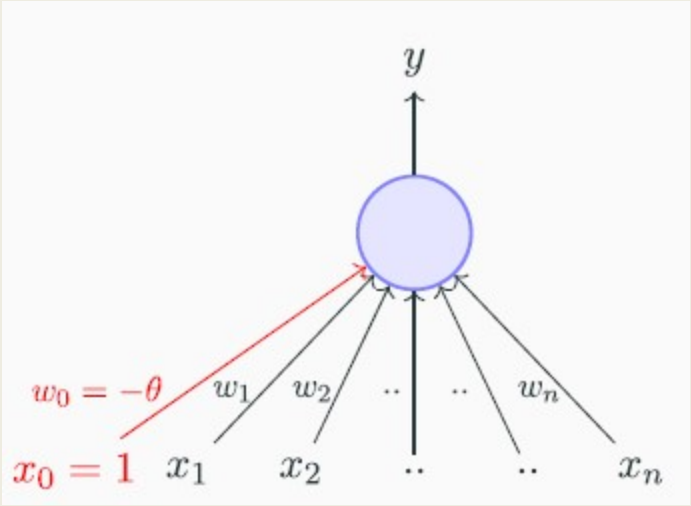


Recap

Rewriting the above

$$y = 1 \quad if \sum_{i=1}^{n} w_i * x_i - \theta \geq 0$$

$$y = 0 \quad if \sum_{i=1}^{n} w_i * x_i - \theta < 0$$



A more accepted convention

$$y = 1 \quad if \sum_{i=0}^{n} w_i * x_i \geq 0$$

$$y = 0 \quad if \sum_{i=0}^{n} w_i * x_i < 0$$

where $x_0 = 1 \quad w_0 = -\theta$

Recap

- McCulloch Pitts Neuron Model

$$y = 1 \quad if \sum_{i=0}^{n} x_i \geq 0$$

$$y = 0 \quad if \sum_{i=0}^{n} x_i < 0$$

- Perceptron

$$y = 1 \quad if \sum_{i=0}^{n} w_i * x_i \geq 0$$

$$y = 0 \quad if \sum_{i=0}^{n} w_i * x_i < 0$$

Learning Algorithm

- Principled approach for learning these weights and threshold
- Apart from implementing boolean functions what can a perceptron be used for ??
- Interest is to classify (binary classifier)

Learning Algorithm

- In other words, perceptron has to find a optimal equation of this separating plane [or to find the values of w_0, w_1, w_2, w_3, w_4

P ← *inputs with Label* 1

N ← *inputs with Label* 0

Initialize w randomly

While !convergence do // algorithm converges when all the inputs are classified correctly

Learning Algorithm

P $\leftarrow$ *inputs with Label* 1

N $\leftarrow$ *inputs with Label* 0

Initialize w randomly

While !convergence do // algorithm converges when all the inputs are classified correctly

Pick random $x \in P \cup N$

if $x \in P$ and $\sum_{i=0}^{n} w_i * x_i < 0$ then

$w = w + x$

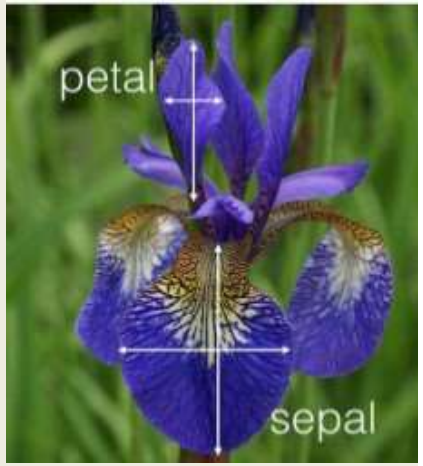
end

if $x \in N$ and $\sum_{i=0}^{n} w_i * x_i \geq 0$ then

$w = w - x$

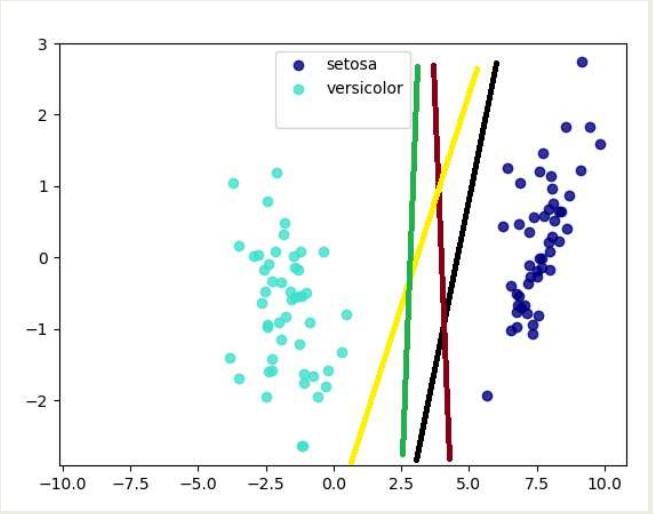
end

Learning Algorithm



	x_1 Sepal length	x_2 Sepal width	x_3 Petal length	x_4 Petal width	Class Label
m_1	5.1	3.5	1.4	0.2	Setosa
m_2	4.9	3.0	1.4	0.2	Setosa
m_3	5.1	3.8	1.9	0.4	Setosa
m_4	7.0	3.2	4.7	1.4	Versicolor
m_5	5.6	2.7	4.2	1.3	Versicolor

- Assume that the data is linearly separable and to design a perceptron to make a decision
- Setosa as class label 1 and versicolor as 0



Learning Algorithm

- Initially consider the weights as follows

$w_o = 0.2 \quad w_1 = 0.38 \quad w_2 = 0.11 \quad w_3 = 0.31 \quad w_4 = 0.09$

Setosa $0.2(x_o) + 0.38(x_1) + 0.11(x_2) + 0.31(x_3) + 0.09(x_4) \geq 0$

Versicolor $0.2(x_o) + 0.38(x_1) + 0.11(x_2) + 0.31(x_3) + 0.09(x_4) < 0$

Epoch I

$0.2(1) + 0.38(5.1) + 0.11(3.5) + 0.31(1.4) + 0.09(0.2) \geq 0 \quad 2.975 \geq 0 \quad setosa$

$0.2(1) + 0.38(4.9) + 0.11(3.0) + 0.31(1.4) + 0.09(0.2) \geq 0 \quad 2.844 \geq 0 \quad setosa$

$0.2(1) + 0.38(5.1) + 0.11(3.8) + 0.31(1.9) + 0.09(0.4) \geq 0 \quad 3.181 \geq 0 \quad setosa$

$0.2(1) + 0.38(7.0) + 0.11(3.2) + 0.31(4.7) + 0.09(1.4) < 0 \quad 4.795 \geq 0 \quad \sout{setosa}$

Versicolor

Update the weight

$if\ x\ \in N \ \text{ and } \sum_{i=0}^{n} w_i * x_i \geq 0 \text{ then} \qquad w = w - x$

Learning Algorithm

Update the weight

$if\ x\ \in N\ \ \text{and}\ \sum_{i=0}^{n} w_i * x_i \geq 0\ \text{then} \qquad w = w - x$

$w_o = w_o - x_0 = 0.2 - 1 = -0.8$

$w_1 = w_1 - x_1 = 0.38 - 7.0 = -6.62$

$w_2 = w_2 - x_2 = 0.11 - 3.2 = -3.09$

$w_3 = w_3 - x_3 = 0.31 - 1.9 = -1.59$

$w_4 = w_4 - x_4 = 0.09 - 1.4 = -1.31$

Updated rule for versicolor

$-0.8(1) + (-6.62)(7.0) + (-3.09)\ (3.2) + (-1.59)(4.7) + (-1.31)(1.4) < 0$

$-66.335 < 0$ Versicolor

$-0.8(1) + (-6.62)(5.6) + (-3.09)\ (2.7) + (-1.59)(4.2) + (-1.31)(1.3) < 0$

$-54.596 < 0$ Versicolor

Learning Algorithm

P $\leftarrow$ *inputs with Label* 1

N $\leftarrow$ *inputs with Label* 0

Initialize w randomly

While !convergence do // algorithm converges when all the inputs are classified correctly

Pick random $x \in P \cup N$

$if\ x \in P$ and $\sum_{i=0}^{n} w_i * x_i < 0$ then

$\Delta w = \lambda * x * (y - \hat{y})$

$w = \Delta w + x$

end

$if\ x \in N$ and $\sum_{i=0}^{n} w_i * x_i \geq 0$ then

$\Delta w = \lambda * x * (y - \hat{y})$

$w = \Delta w - x$

end

λ is the learning rate - controls how quickly the model can adapt to the problem.

Think It !!!

- What about non-boolean inputs?
  - Real valued inputs are allowed in perceptron
- Do we always need to hand code the threshold ?
  - No, we can learn the threshold
- Are all inputs equal ? What if we want to assign more weight (importance) to some inputs?
  - A perceptron allows weights to be assigned to inputs by the user
- What about functions that are not linearly separable?

Linearly inseparable

x_1	x_2	XOR
0	0	0
I	0	I
0	I	I
I	I	0

$$w_o + \sum_{i=1}^{2} w_i x_i < 0$$

$$w_o + \sum_{i=1}^{2} w_i x_i \geq 0$$

$$w_o + \sum_{i=1}^{2} w_i x_i \geq 0$$

$$w_o + \sum_{i=1}^{2} w_i x_i < 0$$

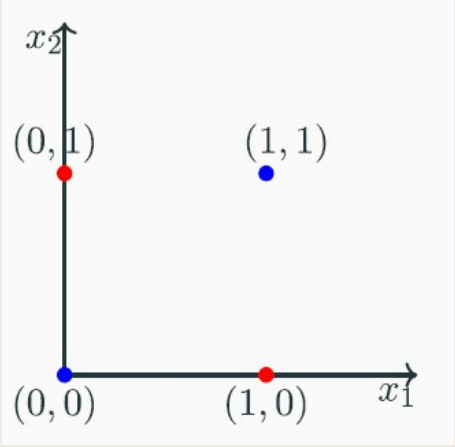
$$w_0 + w_1 0 + w_2 0 < 0 \implies w_o < 0$$

$$w_0 + w_1 1 + w_2 0 \geq 0 \implies w_1 \geq -w_0$$

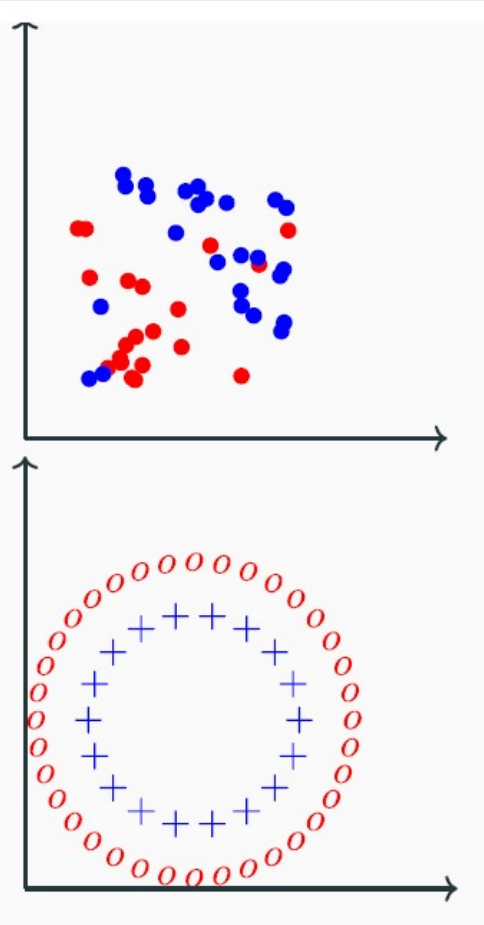
$$w_0 + w_1 0 + w_2 1 \geq 0 \implies w_2 \geq -w_0$$

$$w_0 + w_1 1 + w_2 1 < 0 \implies w_1 + w_2 < -w_0$$

The fourth condition contradicts the 2 and 3



Linearly inseparable



- Most real world data is not linearly separable and will contains some outliers
- Sometimes there may not be any outliers but still the data may not be linearly separable
- Need computational units (model) to deal with such data
- Single perceptron cannot deal with such data, network of perceptrons can indeed deal with such data.

Boolean functions (Detail)

- How many Boolean can be derived from 2 binary inputs ??? 2^{2^n}

x_1	x_2	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}	f_{15}	f_{16}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

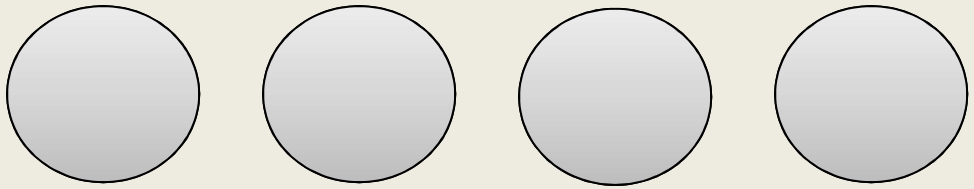
Boolean functions (Detail)

The detailed description of Boolean functions for two variables

Function	x	0	0	1	1
	y	0	1	0	1
Constant 0	0	0	0	0	0
And	$x \cdot y$	0	0	0	1
x And Not y	$x \cdot \bar{y}$	0	0	1	0
x	x	0	0	1	1
Not x And y	$\bar{x} \cdot y$	0	1	0	0
y	y	0	1	0	1
Xor	$x \cdot \bar{y} + \bar{x} \cdot y$	0	1	1	0
Or	$x + y$	0	1	1	1
Nor	$\overline{x + y}$	1	0	0	0
Equivalence	$x \cdot y + \bar{x} \cdot \bar{y}$	1	0	0	1
Not y	$\bar{y}$	1	0	1	0
If y then x	$x + \bar{y}$	1	0	1	1
Not x	$\bar{x}$	1	1	0	0
If x then y	$\bar{x} + y$	1	1	0	1
Nand	$\overline{x \cdot y}$	1	1	1	0
Constant 1	1	1	1	1	1

Network of Perceptrons

- Assume True = +1 and False = -1
- Consider 2 inputs (x_1 and x_2) and 4 perceptrons

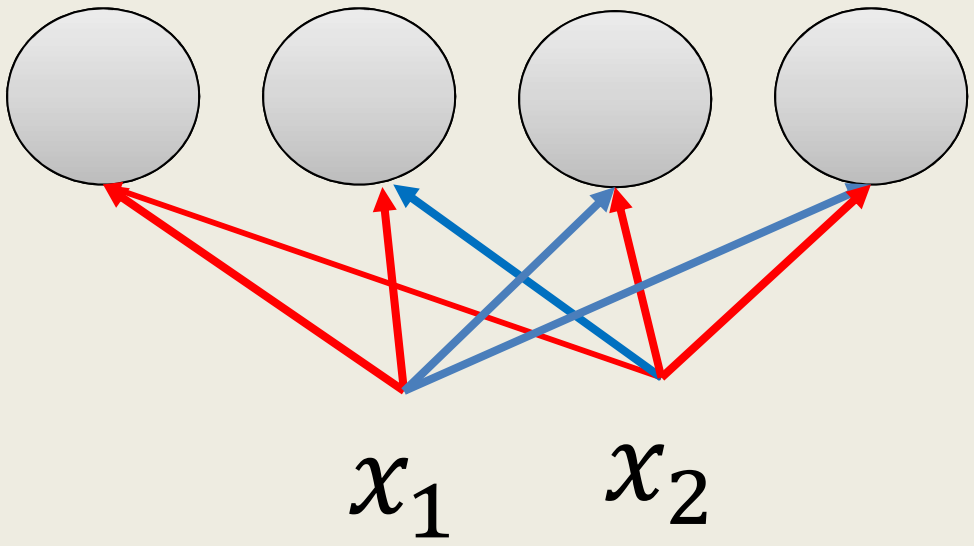


x_1 x_2

- Each input is connected to all the 4 perceptrons with specific weights

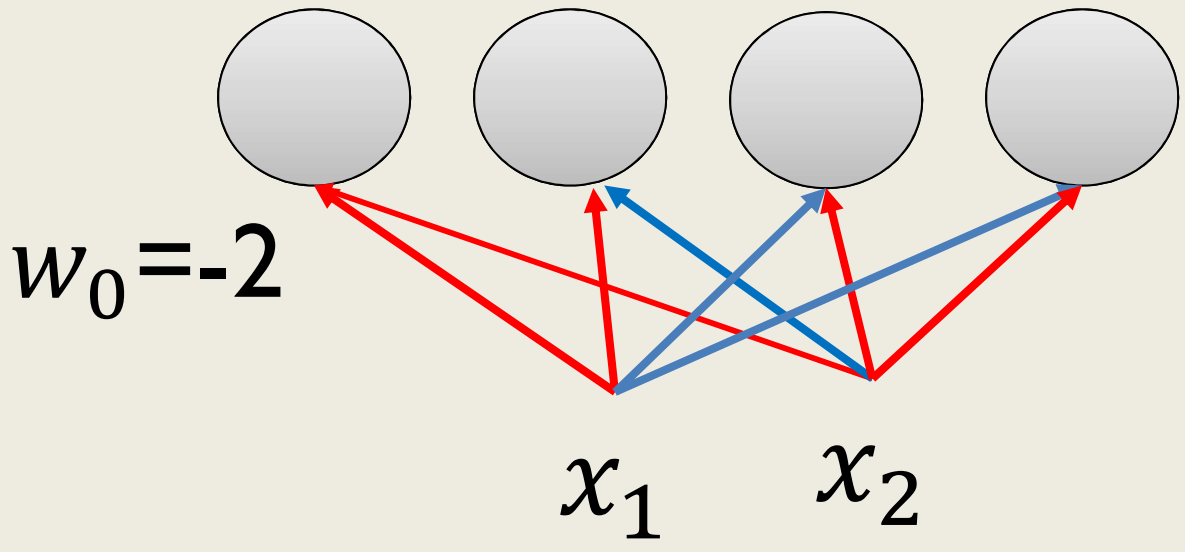
Network of Perceptrons

- Each input is connected to all the 4 perceptrons with specific weights
- Red edge indicates $w = -1$
- Blue edge indicates $w = +1$



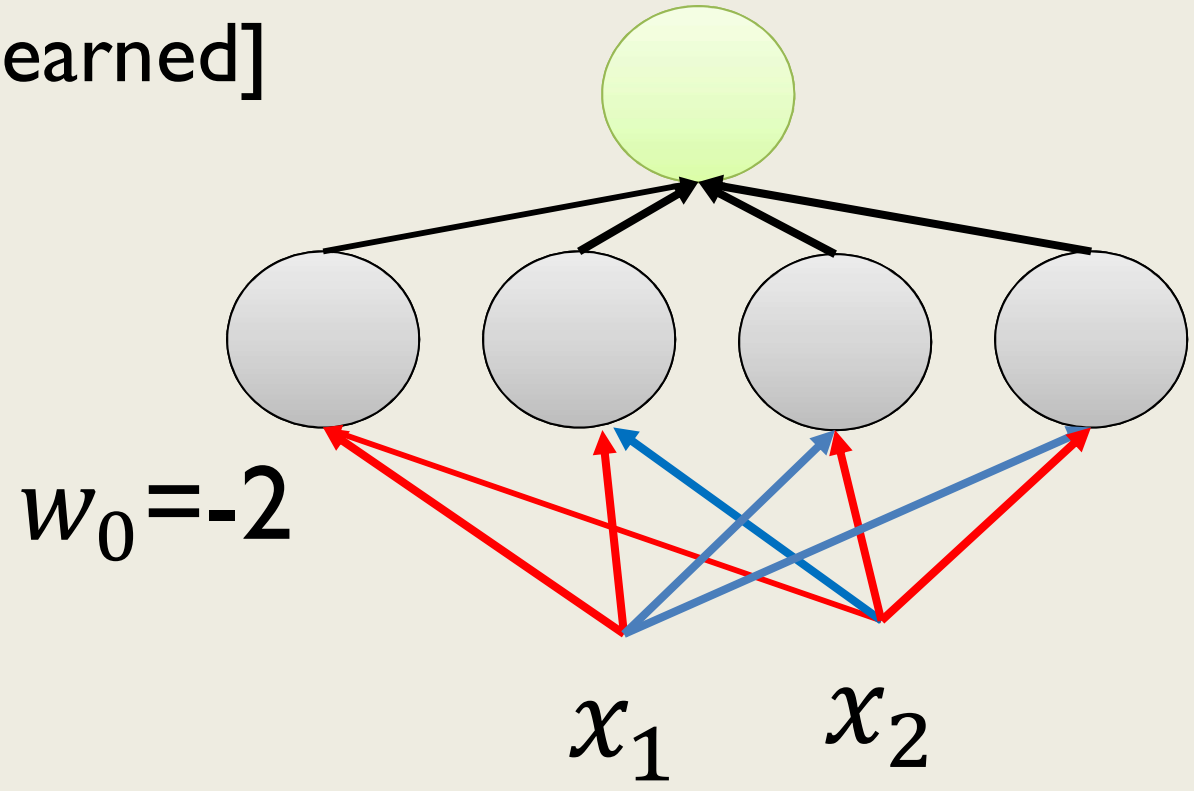
Network of Perceptrons

- Red edge indicates $w = -1$
- Blue edge indicates $w = +1$
- Bias (w_0) of each perceptron is -2



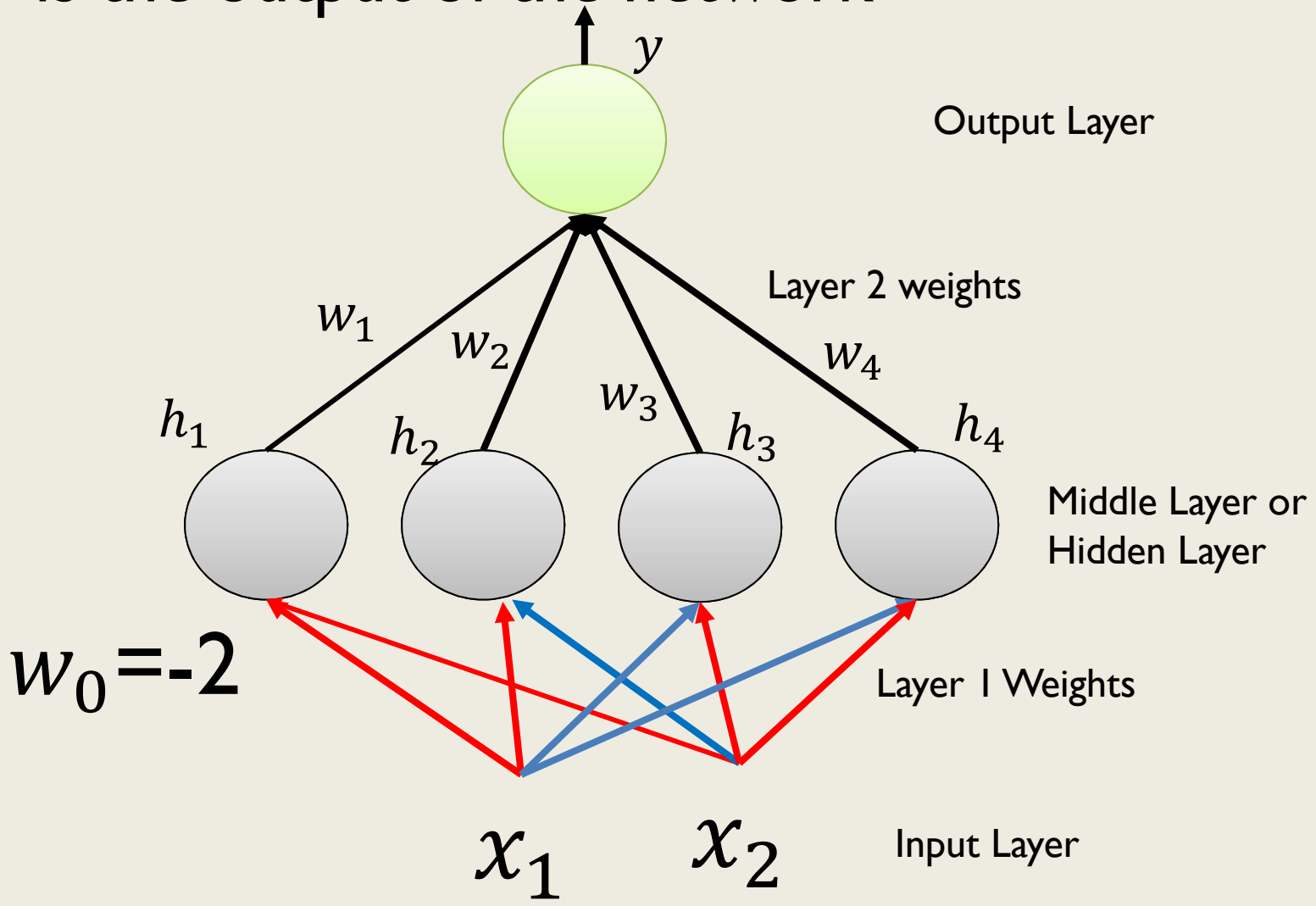
Network of Perceptrons

- Bias (w_0) of each perceptron is -2
- Each perceptron is connected to an output perceptron by weights [which need to be learned]



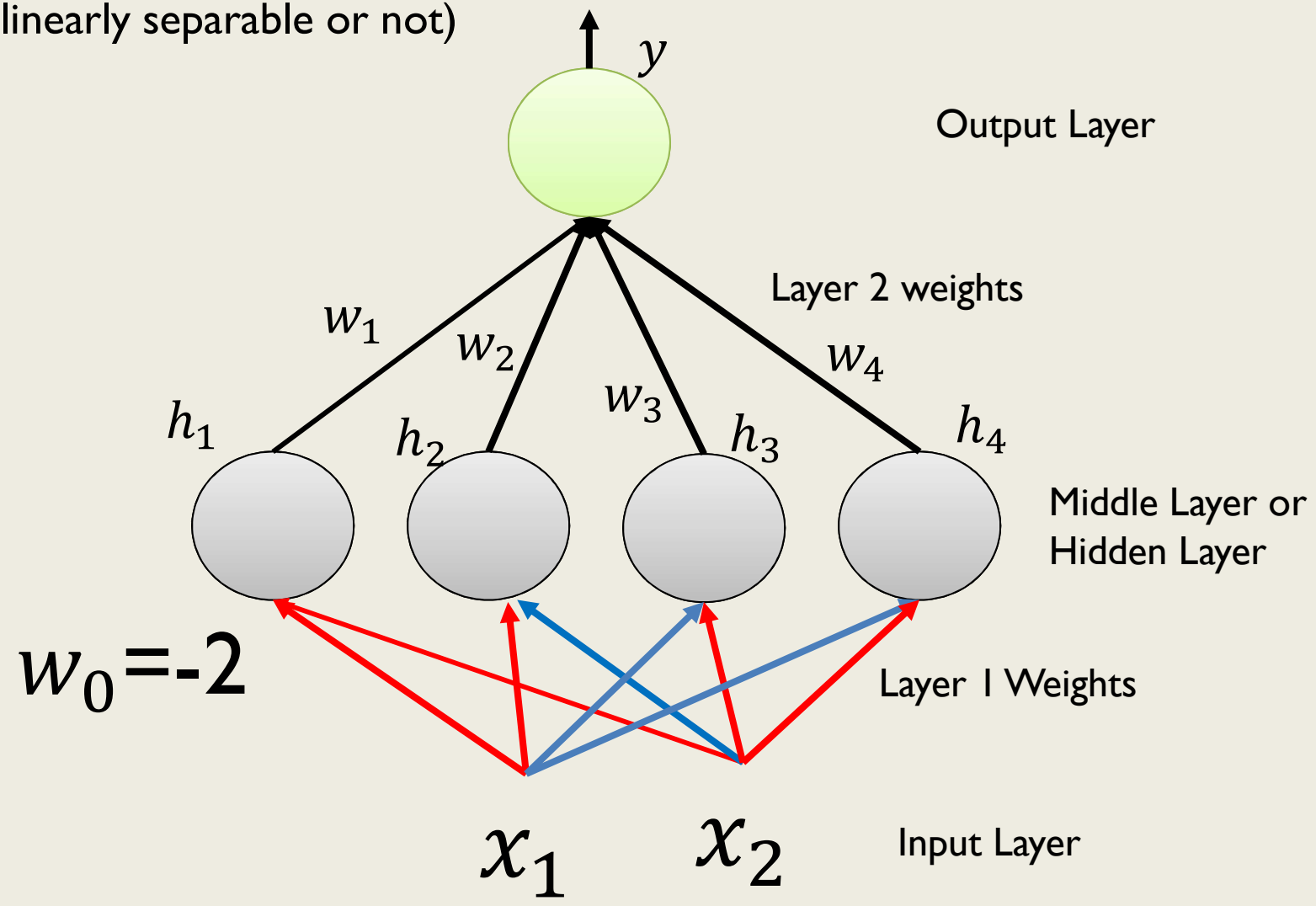
Network of Perceptrons

- y is the output of the network



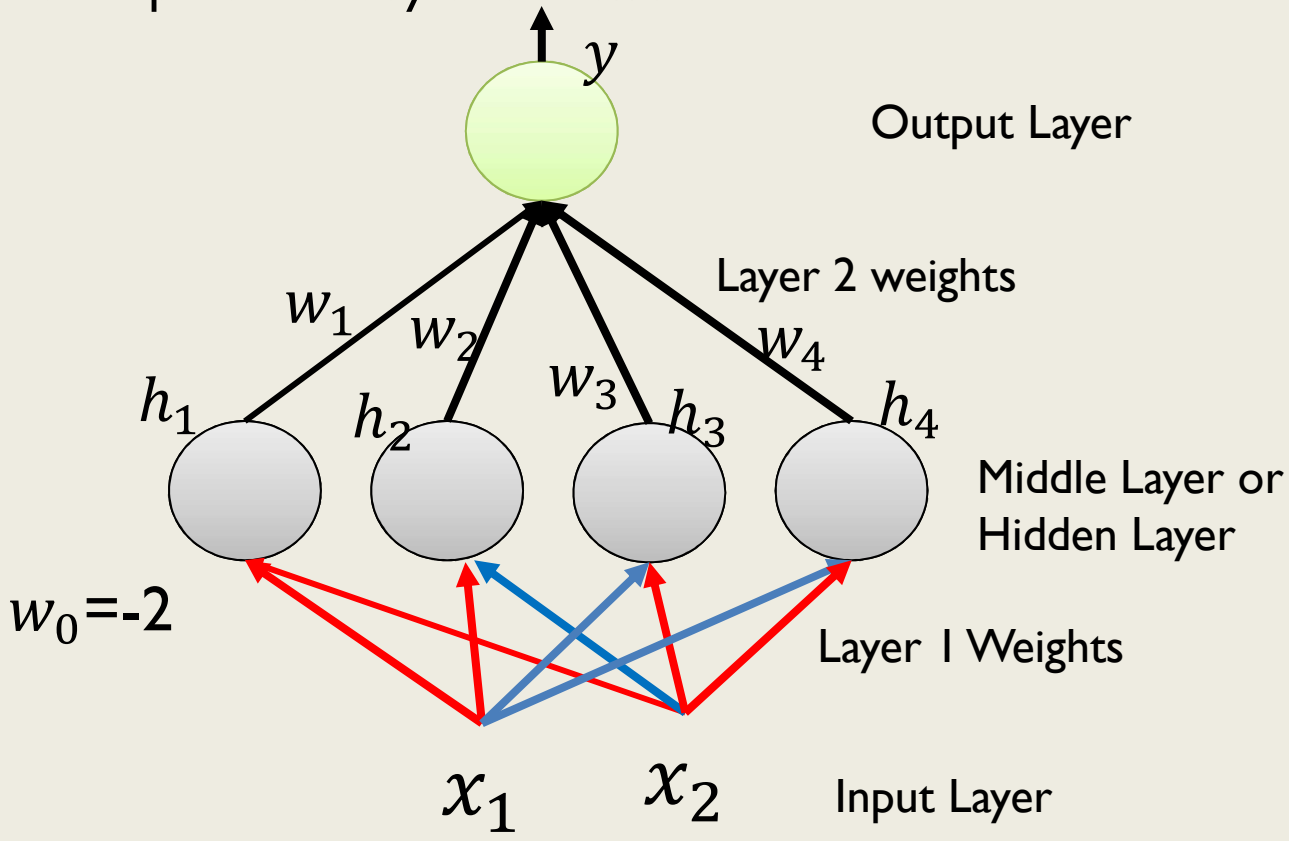
Network of Perceptrons

- Let us claim this network can be used to implement any Boolean function (linearly separable or not)



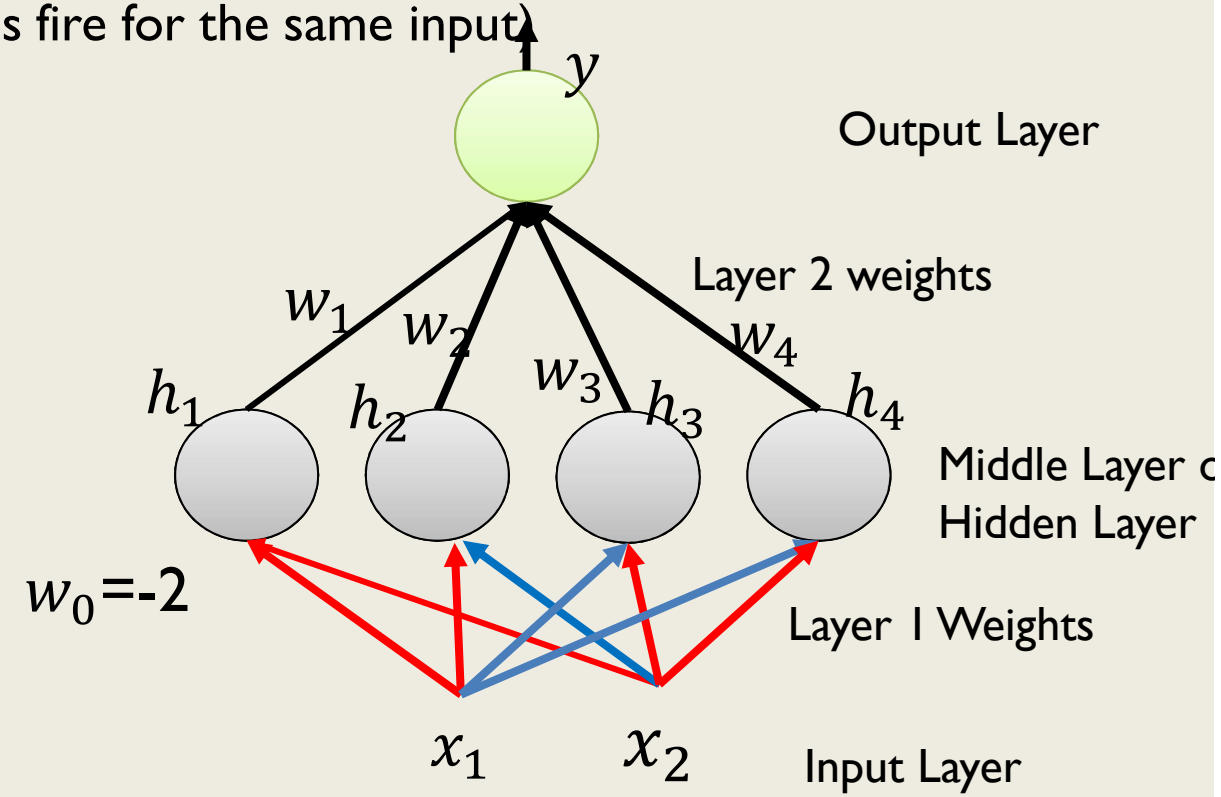
Network of Perceptrons

- Let us claim this network can be used to implement any Boolean function (linearly separable or not)
- In other words, let us find w_1, w_2, w_3, w_4 such that the truth table of any Boolean function can be represented by this network

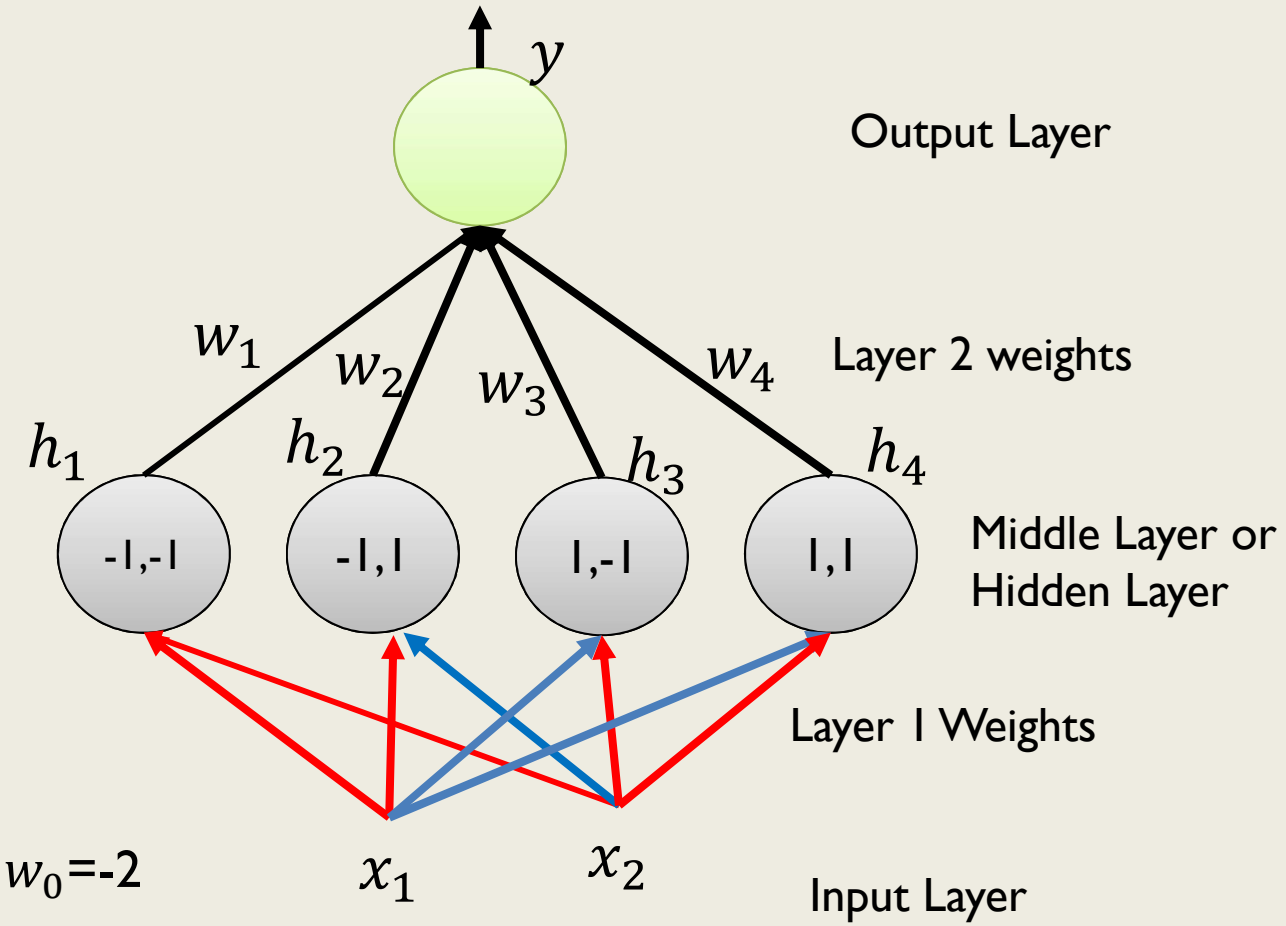


Network of Perceptrons

- Let us claim this network can be used to implement any Boolean function (linearly separable or not)
- In other words, let us find w_1, w_2, w_3, w_4 such that the truth table of any Boolean function can be represented by this network
- Consider each perceptron in the middle layer fires only for a specific input (and no two perceptrons fire for the same input)

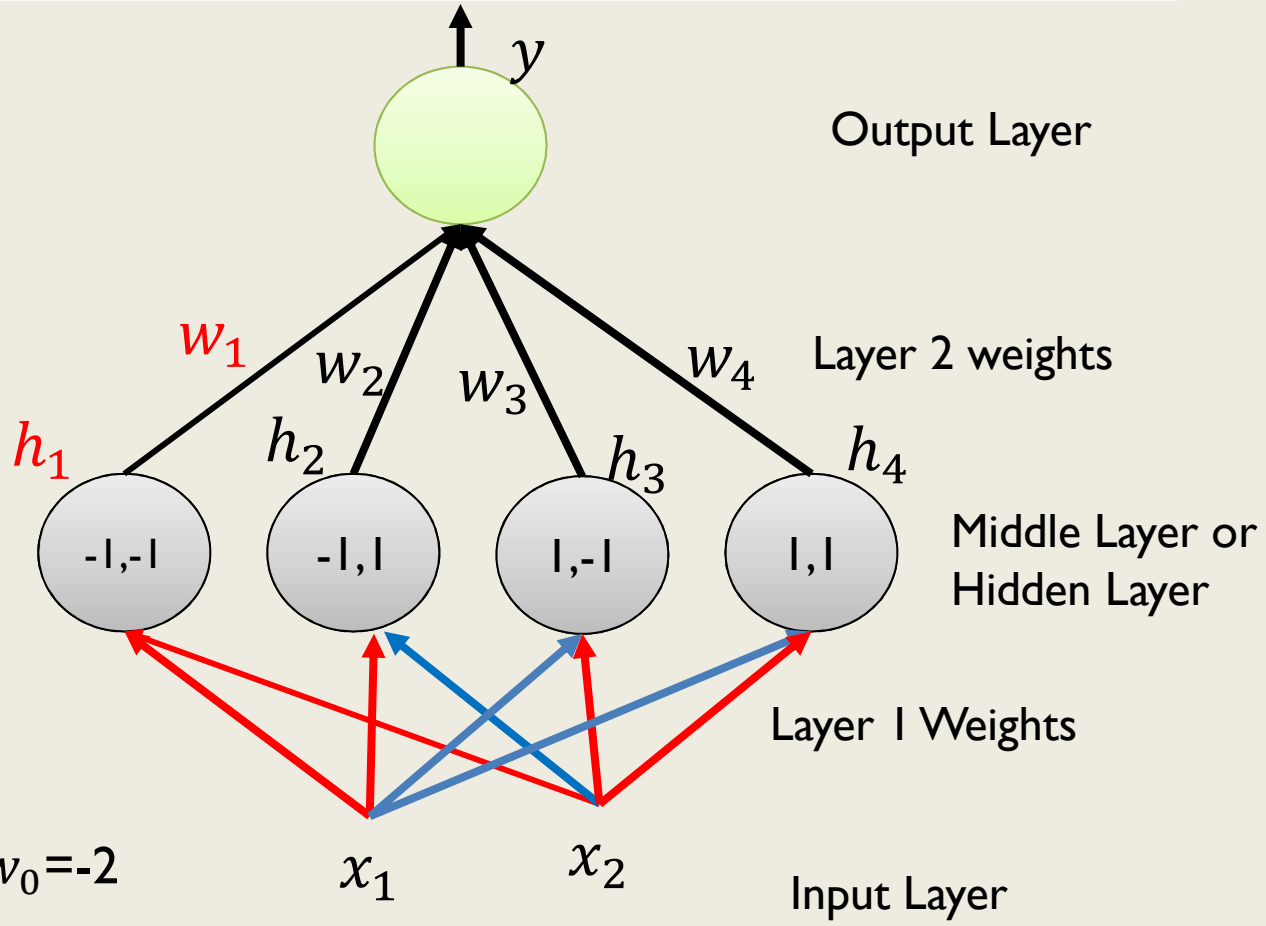


Network of Perceptrons



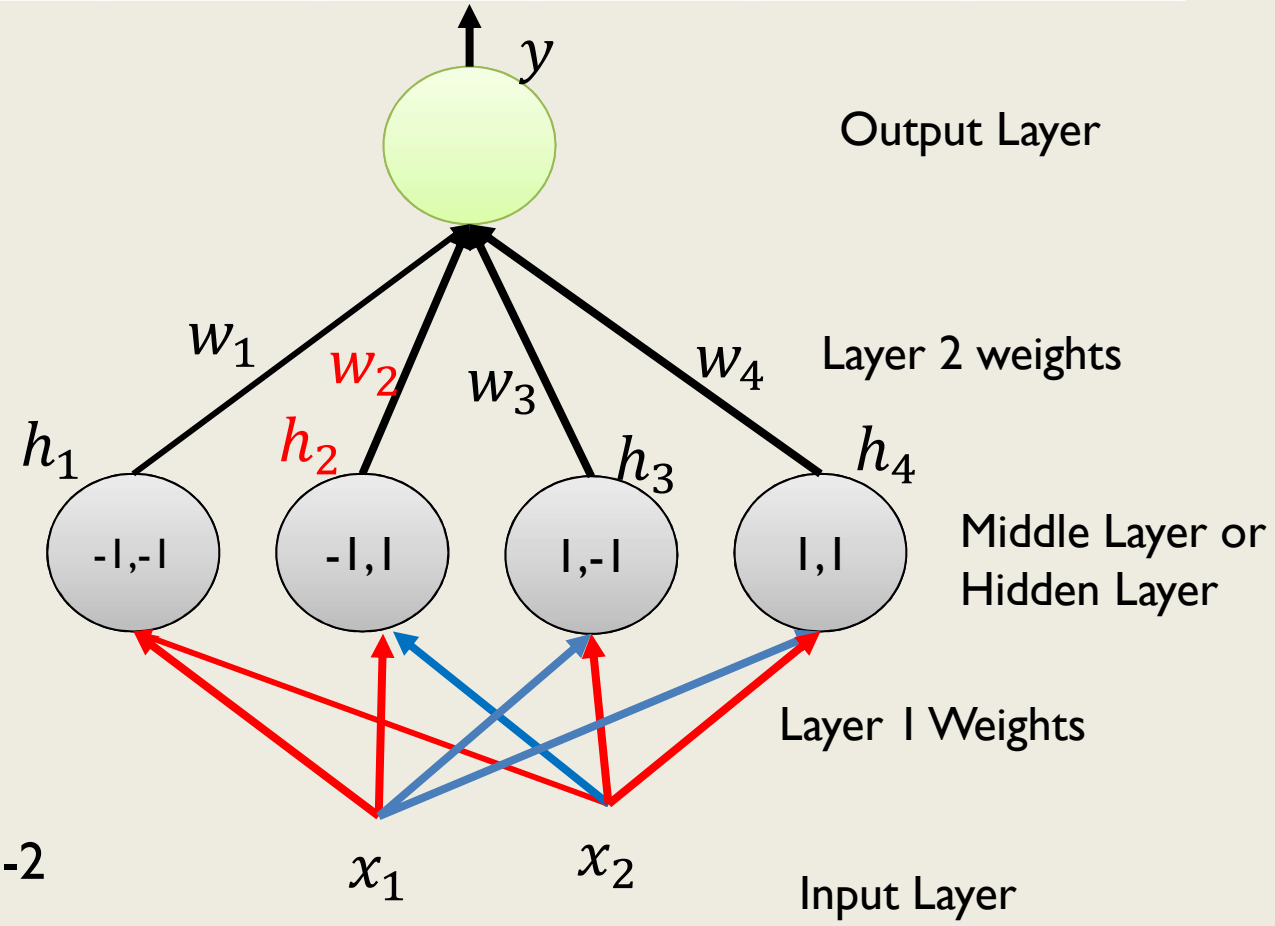
Network of Perceptrons

x_1	x_2	XOR	h_1	h_2	h_3	h_4	$\sum_{i=1}^{4} w_i h_i$
F	F	F	I	0	0	0	w_1



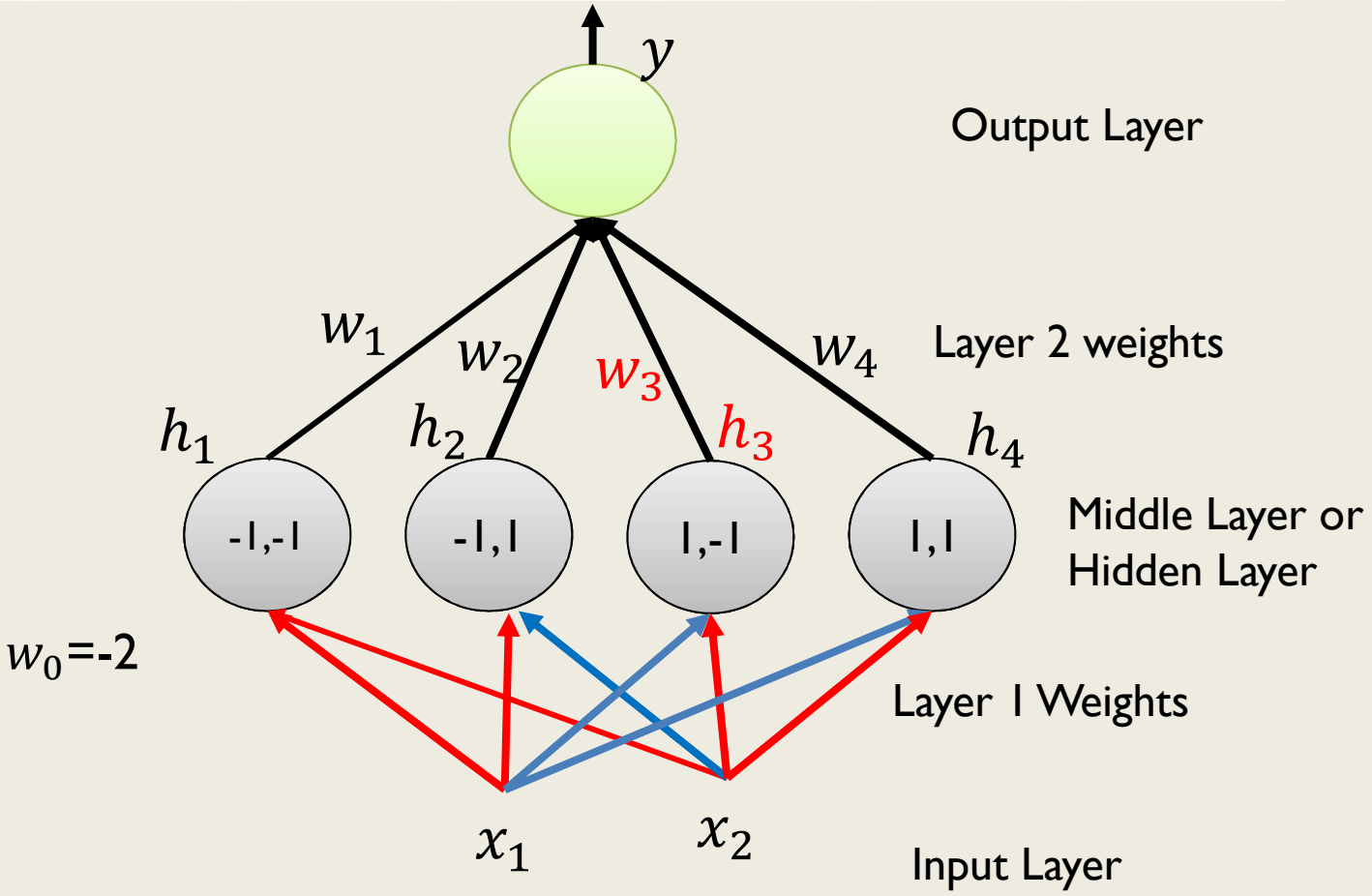
Network of Perceptrons

x_1	x_2	XOR	h_1	h_2	h_3	h_4	$\sum_{i=1}^{4} w_i h_i$
F	T	T	0	I	0	0	w_2



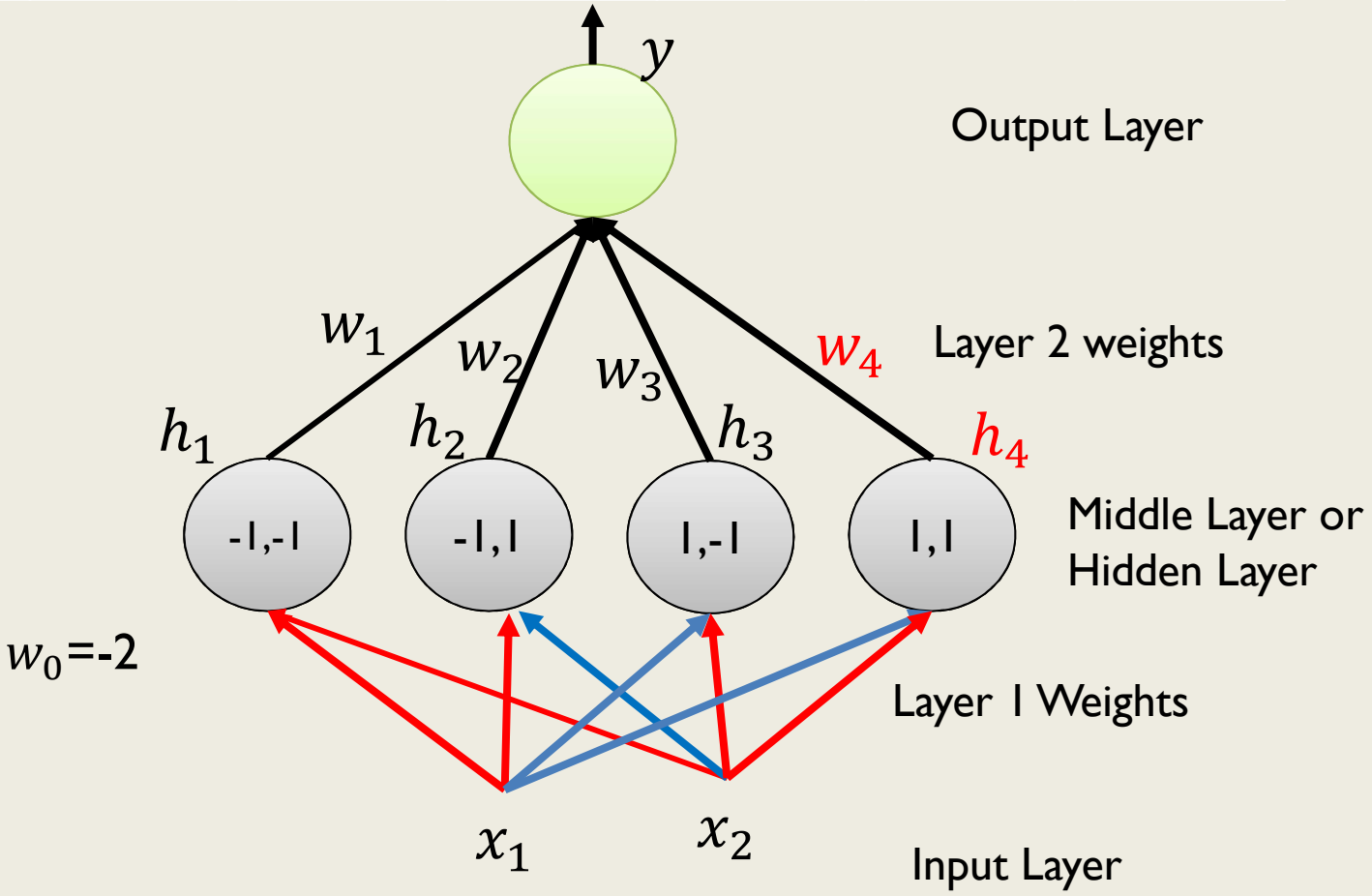
Network of Perceptrons

x_1	x_2	XOR	h_1	h_2	h_3	h_4	$\sum_{i=1}^{4} w_i h_i$
T	F	T	0	0	1	0	w_3



Network of Perceptrons

x_1	x_2	XOR	h_1	h_2	h_3	h_4	$\sum_{i=1}^{4} w_i h_i$
T	T	F	0	0	0	I	w_4



Network of Perceptrons

x_1	x_2	XOR	h_1	h_2	h_3	h_4	$\sum_{i=1}^{4} w_i h_i$
F	F	F	I	0	0	0	w_1
F	T	T	0	I	0	0	w_2
T	F	T	0	0	I	0	w_3
T	T	F	0	0	0	I	w_4

- The output y will fire if $\sum_{i=1}^{4} w_i h_i \geq w_0$
- This results in four conditions to implement XOR :

$$w_1 < w_0 \quad w_2 \geq w_0 \quad w_3 \geq w_0 \quad w_4 < w_0$$